ENGINEERING HIRING CHALLENGE

PROBLEM STATEMENT

Field	Details
Format	Take-home — work independently within the window
Input	Raw anonymised CCTV footage — this is where you start
Output	A working, containerised Store Intelligence API with live metrics
AI Policy	Fully open-book — all AI tools are permitted and expected
Scoring	Automated correctness tests + contextual follow-up questions on your code
Submission	Git repo link + DESIGN.md + CHOICES.md

Read This First

This challenge is end-to-end by design. You start with raw camera footage and you finish with a working analytics API. There is no pre-processed dataset, no skeleton code, and no guardrails on how you build the pipeline in between. **Every architectural decision from frame to API response is yours to make.**

We are not testing whether you know a specific library. We are testing whether you can **decompose a real problem, pick the right tools, build something that works, and explain every decision you made.**

---

1. The Business Problem

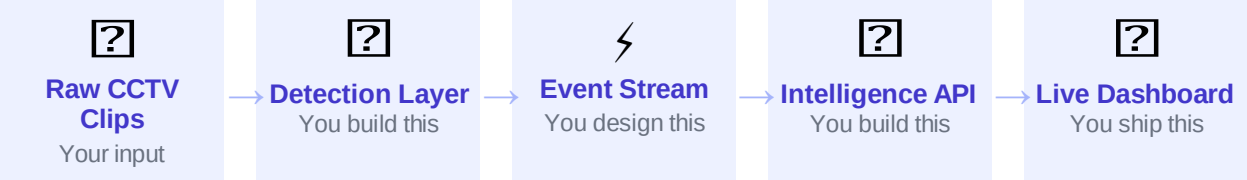
A specialty retail chain — **Apex Retail** — operates 40 physical stores across 8 cities. Their online channel has mature analytics: every session, click, and drop-off is tracked in real time. Their offline stores are a complete data blind spot.

Your job is to build system — starting from the raw camera footage.

---

2. What You Are Building

You are building a complete pipeline — from raw video to live store analytics. The diagram below shows the full system. Every stage is yours to design and implement:



Stage	Your Responsibility	Constraints
1 · Detection Layer	Process the CCTV clips. Detect people. Track movement. Determine direction (entry vs exit). Assign a per-session visitor token.	Use any model or library. Output must be structured events.
2 · Event Stream	Define your event schema. Emit structured events from the detection layer into your ingest pipeline.	Schema must support the analytics queries in Stage 3.
3 · Intelligence API	Ingest events, compute real-time metrics, detect anomalies, expose queryable endpoints.	Must run via docker compose up. API must be production-aware.
4 · Live Dashboard	Show at least one metric updating in real time as events flow in from the detection layer.	Terminal output acceptable. Web UI scores higher.

---

3. The Dataset

3.1 What You Receive

You will receive a ZIP archive containing:

- **CCTV clips** (Entry camera, Main floor camera, Billing area camera)
- **store_layout plan** — zone definitions for each store
- **pos_transactions.csv** — timestamped POS transaction records (store ID, amount, timestamp — no customer identity)
- **sample_events.jsonl** — example events in the expected output schema to help you validate your detection layer

3.2 Video Clip Specifications

Property	Detail
Duration	20 minutes per clip per camera angle (1 hour total per store)
Cameras	3 angles: Entry/Exit threshold · Main floor zone coverage · Billing counter area
Resolution	1080p, 15fps — representative of typical retail CCTV infrastructure
Anonymisation	Full-face blur applied to every frame. Store name and branding masked. No audio.
Lighting	Includes variation: natural light, fluorescent, mixed — mirrors real conditions
Edge cases	Clips intentionally include: groups entering together, staff movement, re-entry, crowded billing, empty periods
Licence	Challenge use only. Must not be published, trained on, or redistributed.

3.3 Known Challenges in the Footage

The clips are realistic — they include the same edge cases you would encounter in a production deployment. Handling them is part of the challenge:

Edge Case	Description	Why It Matters
Group entry	2–4 people entering simultaneously through the same door	Detection must count individuals, not groups
Staff movement	Store staff (identifiable by uniform) move through all zones regularly	Staff must be excluded from customer metrics
Re-entry	Customers who step outside and return — same person, new visit?	Re-entry inflation is a known vendor problem you are solving
Partial occlusion	People partially obscured by displays or other customers	Detection confidence must degrade gracefully, not fail silently
Billing queue buildup	Queue forms, deepens, and partially disperses during the billing clip	Queue depth tracking and abandonment detection

Empty store periods	Some clips include 5–10 minute windows with no customers	Your API must handle zero-traffic correctly — not crash or return null
Camera angle overlap	The floor camera partially overlaps with the entry camera field of view	Cross-camera deduplication — same person should not be double-counted

3.4 POS Transactions

pos_transactions.csv contains records in the format:

POS Schema

store_id, transaction_id, timestamp, basket_value_inr
STORE_BLR_002, TXN_00441, 2026-03-03T14:38:12Z, 1240.00
STORE_BLR_002, TXN_00442, 2026-03-03T14:41:55Z, 680.00

Your pipeline must correlate POS transactions with visitor sessions to compute conversion rate. There is no customer_id in the POS data — correlation is done by **time window + store**. A visitor who was in the billing zone in the 5-minute window before a transaction timestamp counts as a converted visitor for that session.

---

4. What to Build

Part A — Detection Pipeline [30 points]

The Goal

Process the CCTV clips and produce a stream of structured behavioural events. Your detection pipeline is the foundation — everything else depends on the quality of what it emits.

You choose the model, framework, and architecture. The output must be structured events in the schema below. Use any combination of tools:

- Object detection models: YOLOv8, YOLOv9, RT-DETR, MediaPipe, or any other
- Tracking: ByteTrack, DeepSORT, StrongSORT, or custom
- Re-ID: any OSNet / torchreid model, or a distance-based approach using bounding box trajectory
- LLMs / VLMs: use them for zone classification, staff detection, or anything else you find useful

Required Output Schema

Event Schema (your pipeline must emit this)

```
{
  "event_id":    "uuid-v4",              // you generate this — must be globally unique
  "store_id":    "STORE_BLR_002",        // from store_layout.json
  "camera_id":   "CAM_ENTRY_01",         // which camera produced this event
  "visitor_id":  "VIS_c8a2f1",           // your Re-ID token — unique per visit session
  "event_type":  "ZONE_DWELL",           // see catalogue below
  "timestamp":   "2026-03-03T14:22:10Z", // ISO-8601 UTC — derived from clip + frame offset
  "zone_id":     "SKINCARE",             // null for ENTRY / EXIT events
  "dwell_ms":    8400,                   // duration; 0 for instantaneous events
  "is_staff":    false,                  // your model must classify this
  "confidence":  0.91,                   // your detection confidence — do not suppress low-conf events
  "metadata": {
    "queue_depth":  null,                // integer; you populate for BILLING_QUEUE_JOIN
    "sku_zone":     "MOISTURISER",       // zone label from store_layout.json
    "session_seq":  5                    // ordinal position of this event in visitor session
  }
}
```

Event Type Catalogue

Event Type	When to Emit It	Notes
ENTRY	Visitor crosses entry threshold — inbound direction	Starts a new session; assign new visitor_id
EXIT	Visitor crosses entry threshold — outbound direction	Closes the session
ZONE_ENTER	Visitor enters a named zone	Zone names from store_layout.json
ZONE_EXIT	Visitor leaves a named zone	
ZONE_DWELL	Visitor has been in zone continuously for 30+ seconds	Emit every 30s of continued dwell
BILLING_QUEUE_JOIN	Visitor enters billing zone while queue_depth > 0	Set queue_depth in metadata
BILLING_QUEUE_ABANDON	Visitor leaves billing zone before a POS transaction follows	Requires POS correlation
REENTRY	Same visitor_id detected after a prior EXIT	Your Re-ID system must catch this

Detection Scoring Criteria

Criterion	What We Evaluate
Entry/exit count accuracy	How close are your entry and exit counts to ground truth on the held-out clip? (Ground truth provided post-submission for self-evaluation)
Staff exclusion	Are staff events correctly flagged is_staff=true and excluded from customer metrics?
Re-entry handling	Does the same physical person re-entering produce a REENTRY event rather than a second ENTRY?
Group handling	When 3 people enter together, does the pipeline emit 3 ENTRY events or 1?
Confidence calibration	Are low-confidence detections flagged rather than silently dropped or falsely elevated?
Schema compliance	Do all emitted events validate against the schema? Are event_ids unique? Are timestamps correct?

Part B — Intelligence API [35 points]

The Goal

Build a REST API that ingests the events your detection pipeline emits, computes real-time store analytics, detects operational anomalies, and exposes a queryable intelligence surface.

Endpoint	What It Returns	Key Requirements
POST /events/ingest	Accepts batches of up to 500 events. Validates, deduplicates, stores.	Idempotent by event_id. Partial success on malformed events. Structured error response.
GET /stores/{id}/metrics	Today: unique visitors, conversion rate, avg dwell per zone, queue depth, abandonment rate.	Exclude is_staff=true. Handle zero-purchase stores. Real-time — not cached from yesterday.
GET /stores/{id}/funnel	Conversion funnel: Entry → Zone Visit → Billing Queue → Purchase with counts and drop-off %.	Session is the unit, not raw events. Re-entries must not double-count a visitor.
GET /stores/{id}/heatmap	Zone visit frequency + avg dwell, normalised 0–100, ready for grid heatmap rendering.	Include data_confidence flag if fewer than 20 sessions in window.
GET /stores/{id}/anomalies	Active anomalies: queue spike, conversion drop vs 7-day avg, dead zone (no visits in 30 min).	Severity: INFO / WARN / CRITICAL. Include suggested_action string per anomaly.
GET /health	Service status, last event timestamp per store, STALE_FEED warning if >10 min lag.	Must be accurate — this is what an on-call engineer checks first.

Part C — Production Readiness [20 points]

The API must be built as if it will be operated by a team that did not write it:

- **Containerised:** docker compose up starts everything. No manual steps beyond git clone.
- **Structured logging:** Every request logs: trace_id, store_id, endpoint, latency_ms, event_count (for ingest), status_code.
- **Idempotency:** POST /events/ingest is safe to call twice with the same payload. Tests must verify this.
- **Graceful degradation:** Database unavailable → HTTP 503 with structured body. No raw stack traces in responses.
- **Tests:** Statement coverage >70%. Edge cases: empty store, all-staff clip, zero purchases, re-entry in funnel.
- **README:** Setup complete in 5 commands. Includes how to run the detection pipeline against the clips and feed output into the API.

Part D — AI Engineering [15 points]

This is evaluated for how you used AI — not whether you used it. Depth and intentionality score more than volume:

Deliverable	What We Are Looking For
Prompt blocks in test files	Top of each test file: comment block showing the AI prompt used to generate tests + what you changed afterwards. Format: # PROMPT: ... / # CHANGES MADE: ...
DESIGN.md	Plain-language architecture overview. Section titled 'AI-Assisted Decisions': 2–3 places where an LLM shaped your design — and whether you agreed or overrode it.
CHOICES.md	Three decisions: (1) which detection model and why, (2) event schema design rationale, (3) one API architecture choice. For each: options considered, what AI suggested, what you chose and why.
Detection model choice	In CHOICES.md, explain your model selection. If you used a VLM for any part of the pipeline (e.g. zone classification, staff detection), explain the prompt and evaluate whether it worked.

Part E — Live Dashboard [+10 bonus points]

Run your detection pipeline on a clip in real time (or simulated real time) and show at least one store metric updating live on screen. This can be a terminal dashboard (rich, curses) or a web UI. We are looking for proof that the pipeline and API are genuinely connected, not just batch-processed.

5. Scoring

5.1 Point Breakdown

Part	Dimension	Points
A	Entry/exit count accuracy vs ground truth	10
A	Staff exclusion, re-entry, group handling	10
A	Schema compliance and event quality	10
B	API endpoint correctness (held-out event set)	20
B	Funnel accuracy and session deduplication	10
B	Anomaly detection correctness	5
C	Containerisation + README (acceptance gate)	5
C	Structured logs + health endpoint	5
C	Test coverage and edge case handling	10
D	AI usage depth (prompts, DESIGN.md, CHOICES.md)	15
E	Live dashboard bonus	+10
	Total (without bonus)	100

5.2 Acceptance Gate

A submission is scored only if it passes all of the following:

1. **Runs:** docker compose up starts the API. No manual steps beyond git clone.
2. **Produces events:** The README explains how to run the detection pipeline against the clips and where the output goes.
3. **Ingests:** POST /events/ingest accepts events without a 5xx response.
4. **Responds:** GET /stores/STORE_BLR_002/metrics returns a valid JSON response.
5. **Documents:** DESIGN.md and CHOICES.md both exist and are non-trivial (>250 words each).

Submissions that fail the gate get a 12-hour fix window before scoring begins.

5.3 Contextual Follow-Up Questions

After submission, you receive 5 questions generated from your specific code and CHOICES.md. You answer in a 30-minute async video. The questions are designed so that someone who genuinely built the system answers each in under 2 minutes. Examples:

Examples of the Kind of Questions Asked

6. "You used YOLOv8 for detection. Walk me through what you tried when it struggled with the partial occlusion cases in the billing clip."
7. "Your visitor_id assignment uses bounding box trajectory. What breaks when a customer leaves and a different customer enters from the same direction 3 seconds later?"
8. "Your /funnel endpoint is accurate for the test clips. At 40 live stores sending events in real time, what is the first thing that breaks?"
9. "In CHOICES.md you said you considered using a VLM for zone classification but chose rule-based instead. What would make you change that decision?"

These questions cannot be answered generically — they require you to reason about your own submitted code. This is intentional.

---

6. AI Usage Policy

Use Every Tool You Have

Claude, ChatGPT, Cursor, GitHub Copilot, Gemini — use all of them. AI tools are not just allowed, they are **expected**. We specifically evaluate how you use AI, not whether you avoid it.

What matters: Do you use AI to build something **better**? Do you critique its output? Can you explain and defend every line it helped you write? A candidate who uses AI intelligently to solve the hard parts (detection model selection, schema design, edge case handling) and documents that process scores higher than one who hand-codes boilerplate but ignores it for the interesting parts.

Rewarded	Penalised
Using an LLM to evaluate detection model trade-offs and documenting the comparison	Generic CHOICES.md with no personal reasoning that reads as AI-generated filler
Prompting a VLM to help with zone classification and showing the prompt in DESIGN.md	Test files that cover only the happy path and claim high coverage
Iterating on your detection approach based on AI feedback + your own evaluation	A pipeline that ignores all 7 edge cases in the footage

Explaining in CHOICES.md where AI suggested something you disagreed with	Inability to answer follow-up questions about your own code

7. Submission

7.1 Suggested Repository Structure

Suggested Layout

```
/store-intelligence/
├── pipeline/
│   ├── detect.py          # Main detection + tracking script
│   ├── tracker.py         # Re-ID / tracking logic
│   ├── emit.py            # Event schema + emission
│   └── run.sh             # One command to process all clips → events
├── app/
│   ├── main.py            # FastAPI entrypoint
│   ├── models.py          # Pydantic event schema
│   ├── ingestion.py       # Ingest, dedup
│   ├── metrics.py         # Real-time metric computation
│   ├── funnel.py          # Funnel + session logic
│   ├── anomalies.py       # Anomaly detection
│   └── health.py
├── tests/
│   ├── test_pipeline.py   # Include prompt block header
│   ├── test_metrics.py
│   └── test_anomalies.py
├── docs/
│   ├── DESIGN.md          # Architecture + AI-assisted decisions
│   └── CHOICES.md         # 3 decisions with full reasoning
├── docker-compose.yml
└── README.md
```

The structure is a suggestion — if your architecture dictates something different, explain it in DESIGN.md. We do not penalise deviation from the suggested layout.

7.2 Submission Checklist

- Git repository link (private — invite reviewer handle provided here purplletechchallenge2026@hackerearth.com)
- docker compose up confirmed working on a clean machine before submission
- README.md explains how to run the detection pipeline against the clips

- DESIGN.md includes 'AI-Assisted Decisions' section
- CHOICES.md covers: model selection, schema design, one API decision
- Prompt blocks at top of each test file
- If doing Part E (dashboard): local URL noted in README.md

---

8. North Star

Every component you build connects to a single business metric:

North Star Metric: Offline Store Conversion Rate

Conversion Rate = Visitors who completed a purchase ÷ Total unique visitors in a session window

Every stage of your pipeline either improves the accuracy of this number (detection layer) or makes it actionable (API layer). When you make a design trade-off, ask yourself: does this make the metric more accurate or more useful?

Business Question	Where Your System Answers It
How many customers visited today and how many bought?	Detection accuracy + /metrics conversion_rate
Where in the store are we losing customers?	/funnel drop-off % by stage
Which product zones get attention but not sales?	/heatmap dwell vs /funnel billing stage
Is there a queue building right now?	/anomalies BILLING_QUEUE_SPIKE
Is our conversion rate worse than usual today?	/anomalies CONVERSION_DROP
Is any camera or store feed stale?	/health STALE_FEED warning

---

9. FAQ

Question	Answer

Do I have to use Python?	No — but Python is recommended. The scoring harness has best coverage for FastAPI. Go and Node.js are acceptable. Your detection pipeline can be any language.
Which detection model should I use?	Your choice. YOLOv8 is a common starting point but not required. We are evaluating your reasoning, not your specific model.
Can I use a VLM (GPT-4V, Claude Vision, Gemini) for any part of the pipeline?	Yes. Document what you used it for and how in DESIGN.md.
What storage engine for the API?	Your choice. SQLite is fine. PostgreSQL works. Document it in CHOICES.md.
What if my detection isn't perfect?	That's expected — production CV systems are never perfect. What we evaluate is how you handle uncertainty, confidence thresholds, and edge cases — not a perfect detection rate.
Can I pre-process the clips offline and then replay events into the API?	Yes. The pipeline can be batch or streaming. If you want bonus points on Part E, it needs to be real-time or simulated real-time.
What if I can't finish all parts?	Parts A and B are weighted most heavily. A strong detection pipeline + solid API will score well even without Part D or E.
How are follow-up questions delivered?	Via email within 2 hours of submission. You have 48 hours to record and upload your video responses.
Can I ask clarifying questions?	Yes — email the hiring team. Response SLA: 4 hours (10am–7pm IST, Mon–Sat).